Atividade Integradora – Comunicação de Dados e Aplicações Distribuídas

Grupo: Fernando Grahl Zuffo, Gian Marcos Boaretto, Felipe Castilhos e Luiz Felipe Kauê da Silva.

Tema: Automação Industrial.

Problema: Desenvolver um sistema para monitorar um moinho industrial. O sistema deve identificar sobrecarga por excesso de corrente ou falha no sensor de medição e interromper automaticamente a operação para evitar danos e acidentes.

Justificativa: O monitoramento contínuo reduz paradas inesperadas, evita danos aos equipamentos e aumenta a segurança dos operadores por meio da coleta e processamento de dados em tempo real.

Cenário 1 – Moinho: Operador → Alimentador → Moinho → Sensor de Corrente → CLP → Sistema Supervisório. Se a corrente ultrapassar o limite definido, o CLP desliga o equipamento e registra o evento.

Dados coletados: corrente elétrica, estado dos sensores, status da máquina, horários de falhas e histórico de alarmes.

Usuários: Operador, Líder de Produção, Manutenção Industrial, Técnico de Segurança do Trabalho e Supervisores.

Objetivo: Utilizar comunicação de dados e aplicações distribuídas para monitorar máquinas industriais em tempo real, detectar anomalias e acionar mecanismos automáticos de proteção.

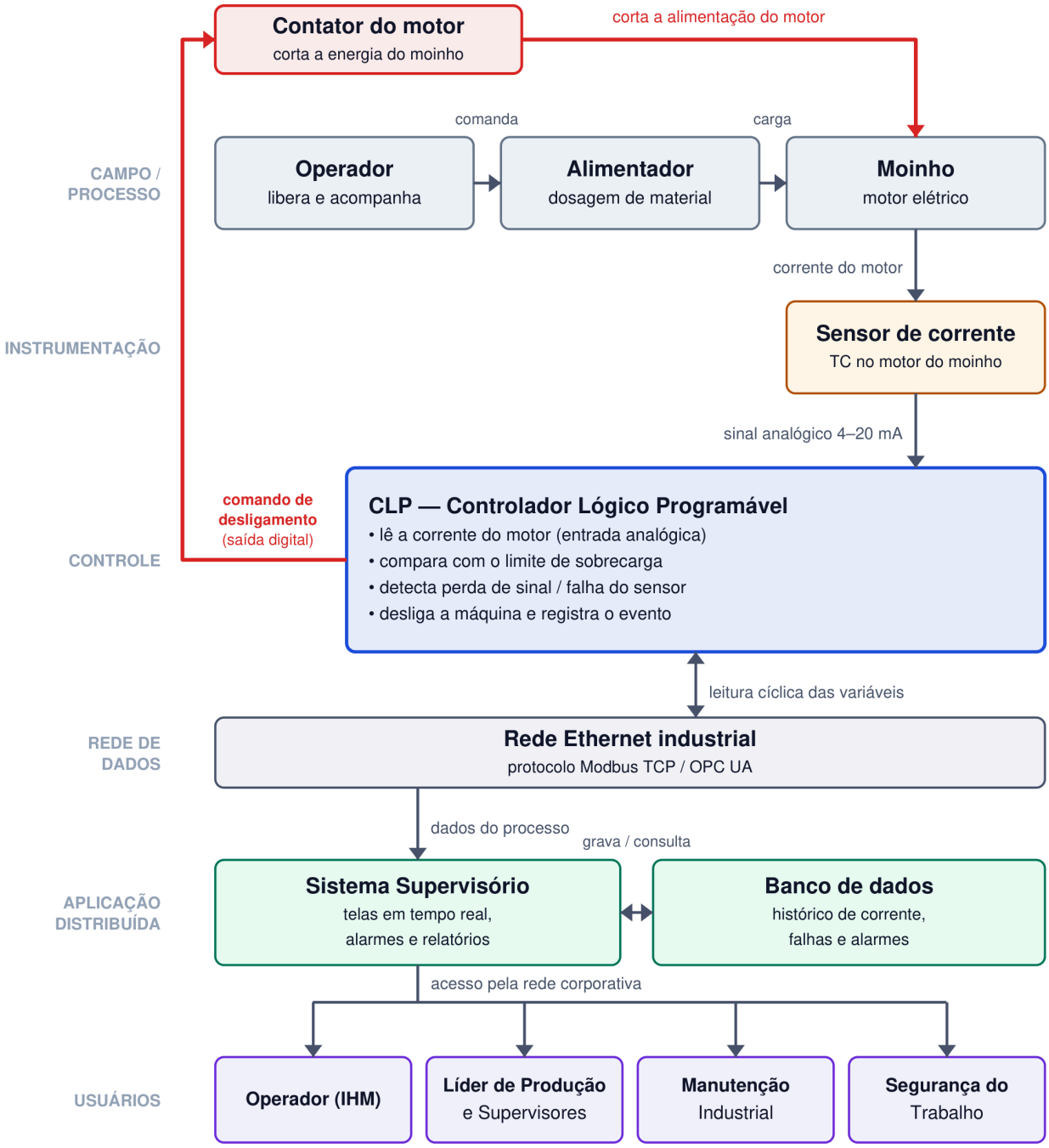


Figura 1 – Arquitetura da solução do Cenário 1, do processo físico até o usuário final. As setas cinzas representam o fluxo de dados; as setas vermelhas representam a malha de proteção, em que o comando de desligamento parte do próprio CLP e abre o contator do motor, sem depender da rede.

Comunicação sem Fio

Além da rede cabeada apresentada no Cenário 1, a arquitetura prevê uma via de comunicação sem fio para a aquisição e a distribuição dos dados de corrente. O caminho da informação passa a ser: Sensor → Wi-Fi → MQTT → CLP / Sistema Supervisório.

O sensor de corrente transmite suas leituras por Wi-Fi para a rede industrial. Na rede, os dados são publicados em um broker MQTT, que organiza as mensagens em tópicos (por exemplo, moinho/corrente e moinho/status) e as entrega aos assinantes interessados: o CLP e o Sistema Supervisório. O supervisório, por sua vez, grava o histórico no banco de dados para consulta posterior.

Esse modelo publish/subscribe reduz a necessidade de cabeamento, facilita a inclusão de novos sensores e permite que vários sistemas consumam a mesma leitura simultaneamente, sem que o sensor precise conhecer seus destinatários.

Independência da rede (segurança): a comunicação sem fio é utilizada para monitoramento, integração e registro histórico — nunca como único caminho de proteção. O CLP continua responsável pela proteção em tempo real, com a leitura direta do sinal 4–20 mA e o comando de desligamento cabeado até o contator do motor. Assim, se o Wi-Fi perder conexão, o broker MQTT ficar indisponível ou houver perda de mensagens, o CLP ainda desliga o moinho por sobrecorrente. A falha da rede sem fio degrada apenas a visibilidade dos dados (telas e histórico), não a proteção da máquina.

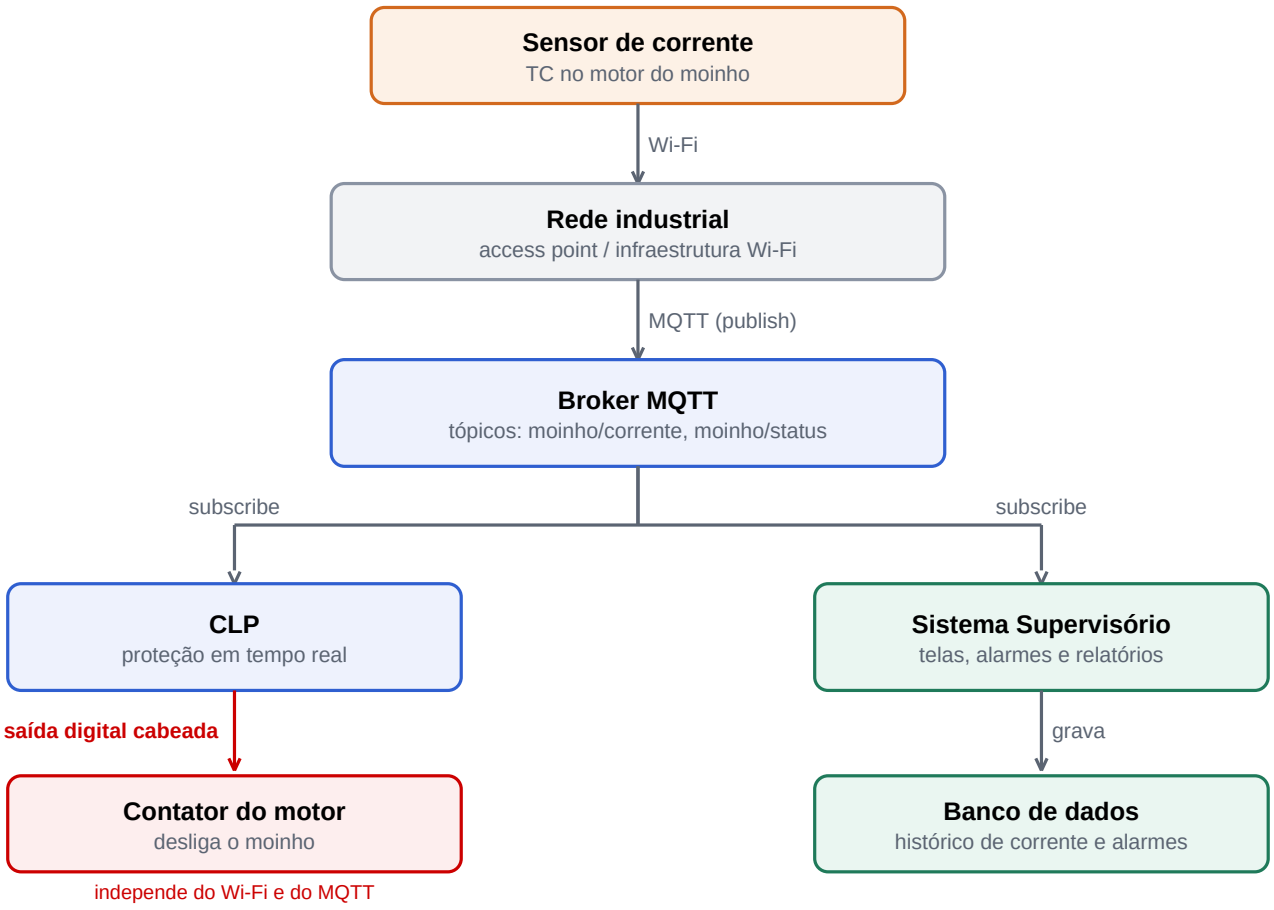


Figura 2 – Comunicação sem fio do Cenário 1. O sensor publica as leituras via Wi-Fi/MQTT e o broker as distribui ao CLP e ao Sistema Supervisório. As setas vermelhas indicam a malha de proteção cabeada, que permanece ativa mesmo com a rede sem fio indisponível.

Comunicação e Consistência do Sistema

Nesta etapa, o grupo implementou um dos fluxos da arquitetura apresentada na Figura 2: a leitura de corrente sai do nó de medição, passa pelo broker MQTT, chega ao Sistema Supervisório, é gravada no banco de dados e aparece na tela do operador.

Código-fonte e evidências: o código da implementação, os scripts de teste, os registros completos e as imagens estão no repositório https://github.com/GianMarcosBoaretto/moinho-mqtt.

Protótipo de bancada: o ESP32 reúne os papéis do sensor e do CLP da Figura 2. Um potenciômetro simula o sensor de corrente e um módulo relé substitui o contator. A botoeira de emergência fica ligada em série com o relé. A proteção continua local, como definido na Figura 2: o ESP32 mede, compara com o limite e abre o relé sem consultar a rede. Para testar sem hardware, um simulador em Python reproduz a mesma lógica e as mesmas mensagens. Uma página web também permite usar o celular no lugar do ESP32.

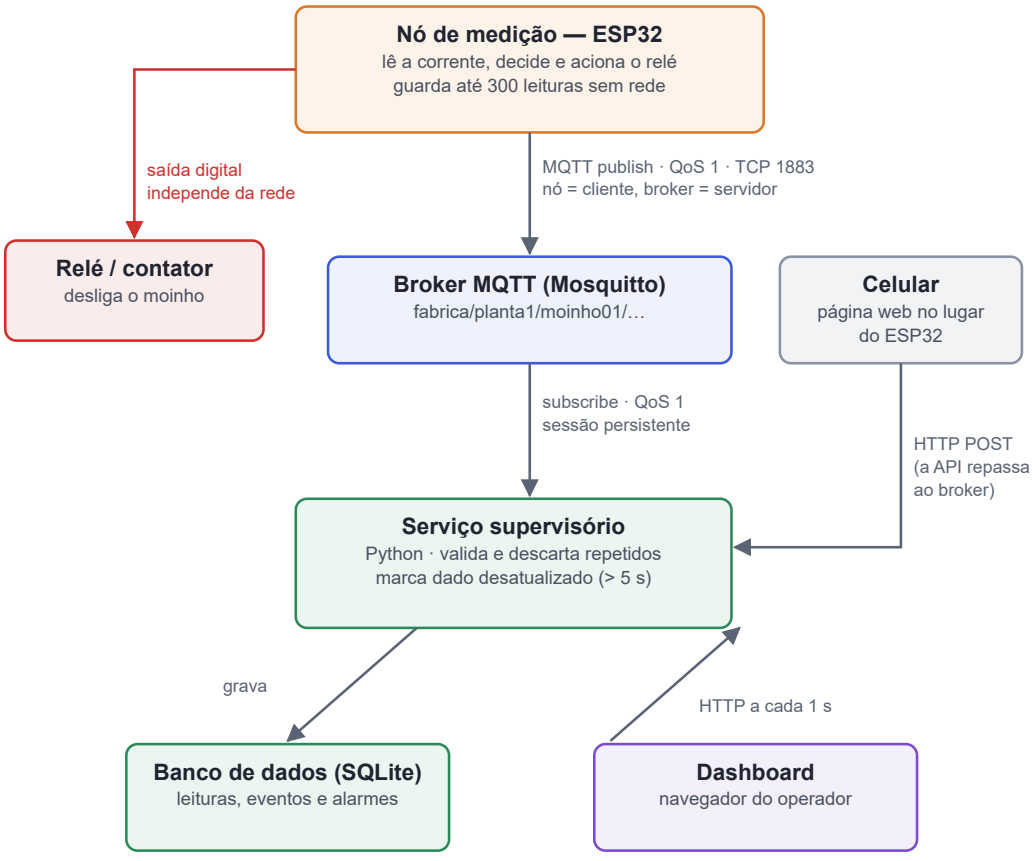


Figura 3 – Fluxo implementado. As setas cinzas representam a comunicação de monitoramento; a seta vermelha representa a proteção, que é decidida no próprio nó e não passa pela rede.

Comunicação entre os componentes: o nó publica as leituras no broker por MQTT sobre TCP, com QoS 1, e o broker confirma o recebimento de cada mensagem. No MQTT, o nó e o supervisório são clientes e só o broker é servidor; por isso os dois não precisam se conhecer e podem cair e voltar de forma independente. O supervisório assina os tópicos, grava cada leitura no SQLite e oferece uma API HTTP, consultada a cada segundo pelo dashboard. O celular não abre conexão MQTT direta, porque o navegador não permite: ele envia a leitura por HTTP e a API publica no broker em nome dele.

Nomeação

Os tópicos seguem a hierarquia `fabrica/<planta>/<equipamento>/<tipo>`:

- `fabrica/planta1/moinho01/corrente` – leitura de corrente, a cada 1 s;
- `fabrica/planta1/moinho01/alarme` – desarme, emergência ou rearme;
- `fabrica/planta1/moinho01/status` – se o nó está conectado e em que estado (mensagem retida no broker).

O identificador do equipamento (`device_id` = moinho01) é o mesmo no tópico, na mensagem e na chave do banco. Cada conexão MQTT tem um `client_id` fixo e único: esp32-moinho01 para a placa, sim-moinho01 para o simulador e supervisorio-planta1 para o supervisório. Dois clientes com o mesmo id derrubam um ao outro no broker, o que o grupo chegou a observar durante os testes. As mensagens são JSON com versão de formato (`schema`):

```
{"schema": 1, "device_id": "moinho01", "seq": 1000000123,
 "ts_origem": "2026-10-14T12:32:56.820Z", "corrente_a": 129.8,
 "estado": "desarmado", "motivo": "sobrecorrente"}
```

O campo `seq` é um contador que só cresce em cada dispositivo. Para não voltar a 1 quando a placa reinicia, ele inclui o número de reinicializações gravado na memória do ESP32.

Sincronização

O horário da medição (`ts_origem`) é gerado no próprio nó, em UTC, com o relógio do ESP32 acertado por NTP. Ao receber a mensagem, o supervisório grava também o horário de chegada (`ts_recebimento`) e a diferença entre os dois, que é o atraso da entrega. Em operação normal o atraso fica em poucos milissegundos; depois de uma falha, as leituras que estavam guardadas chegam com atraso de vários segundos, e é assim que o sistema sabe que elas vieram do buffer.

Todas as consultas e telas ordenam os dados por `ts_origem`, nunca pela ordem de chegada: quando a comunicação volta, leituras antigas chegam misturadas com as novas, e a ordem de chegada mostraria um valor velho como se fosse o atual. O `seq` completa a sincronização: se depois do 41 chega o 44, faltam duas leituras, e o dashboard mostra essa lacuna.

Comportamento em Caso de Falha de Comunicação

O que acontece com os dados durante a falha: a proteção não muda, o nó continua medindo a cada 100 ms e desarmando por sobrecorrente ou falha de sensor. As leituras que não conseguem ser enviadas ficam em um buffer circular de 300 posições no nó, o equivalente a 5 minutos. Se a falha durar mais que isso, as leituras mais antigas são descartadas e a perda aparece como lacuna no histórico. Se só o supervisório cair, o broker guarda as mensagens dele (sessão persistente).

O que o usuário verá: quando a última leitura passa de 5 segundos, o dashboard mostra o aviso "Comunicação perdida há X s — último dado recebido às HH:MM:SS", com a causa provável (supervisório sem acesso ao broker ou nó desconectado). A corrente e o estado continuam na tela, mas em cinza e marcados como valor antigo, e o gráfico mostra o trecho sem dados.

O que acontece quando a comunicação voltar: o nó reconecta e envia o buffer na ordem, com o horário e o número de sequência originais. O supervisório grava cada leitura e descarta as repetidas pela chave (`device_id`, `seq`), já que o QoS 1 pode entregar a mesma mensagem duas vezes. Como tudo é ordenado pelo horário da medição, o histórico é reconstruído no lugar certo, e um desarme que aconteceu durante a falha aparece com o horário em que de fato aconteceu.

Como o sistema identifica que um dado está desatualizado: um dado é considerado desatualizado quando o horário atual menos o `ts_origem` da última leitura passa de 5 segundos, ou seja, cinco leituras seguidas sem chegar. O critério usa o horário da medição e não o de chegada, então uma leitura antiga que acabou de chegar do buffer continua sendo tratada como antiga. O broker também avisa quando o nó desconecta sem se despedir (mensagem de "última vontade", LWT), mas isso não cobre o caso em que o próprio broker cai, por isso o critério principal é a idade do dado.

Replicação e Consistência

A mesma leitura existe em mais de um lugar: no buffer do nó, na fila do broker e no banco do supervisório. A cópia do nó é a de origem; as demais são atualizadas depois, sem que o nó espere por elas (replicação assíncrona). Por isso o banco pode ficar temporariamente atrás do que aconteceu no moinho, e o grupo considera isso aceitável por três motivos. Primeiro, nenhuma decisão de proteção depende do banco. Segundo, ele se acerta sozinho quando a comunicação volta, porque cada leitura tem identificação única, a gravação ignora repetidas e a ordem vem do horário da medição. Terceiro, enquanto ele está atrasado, a tela avisa. É um modelo de consistência eventual.

Decisões e Justificativas

Proteção local: uma sobrecorrente precisa ser cortada em fração de segundo. Se o desarme dependesse de Wi-Fi, broker e supervisório, a proteção deixaria de existir justamente quando a rede falhasse. Sem leitura válida do sensor, o nó também desarma, por segurança.

MQTT com QoS 1: o modelo publish/subscribe permite incluir novos consumidores sem alterar o nó, e o protocolo já oferece confirmação de entrega, sessão persistente, mensagens retidas e LWT. O QoS 1 garante que a mensagem chega ao menos uma vez; somado ao descarte de repetidas no banco, o resultado é o mesmo do QoS 2, com menos mensagens trocadas.

Tecnologias: Mosquitto como broker, Python com FastAPI no supervisório, SQLite como banco e uma página HTML simples como dashboard. São ferramentas leves, que rodam em um único computador de bancada e bastam para demonstrar o fluxo.

Evidências de Funcionamento

O grupo criou um teste automatizado que sobe o broker, o supervisório e o simulador, derruba o broker durante 32 segundos, força uma sobrecorrente nesse intervalo e religa o broker. Das 14 verificações, todas passaram:

- o nó desarmou sem rede;
- o supervisório marcou o dado como desatualizado;
- 31 leituras foram reenviadas do buffer;
- o histórico ficou sem lacunas;
- o desarme foi registrado com o horário real;
- duas mensagens repetidas, enviadas de propósito, foram descartadas.

No repositório estão os registros completos do teste (pasta evidencias/teste-falha-local-2026-10-14) e as imagens em tamanho original (pasta evidencias/prints). As figuras a seguir mostram o dashboard antes, durante e depois de uma falha.

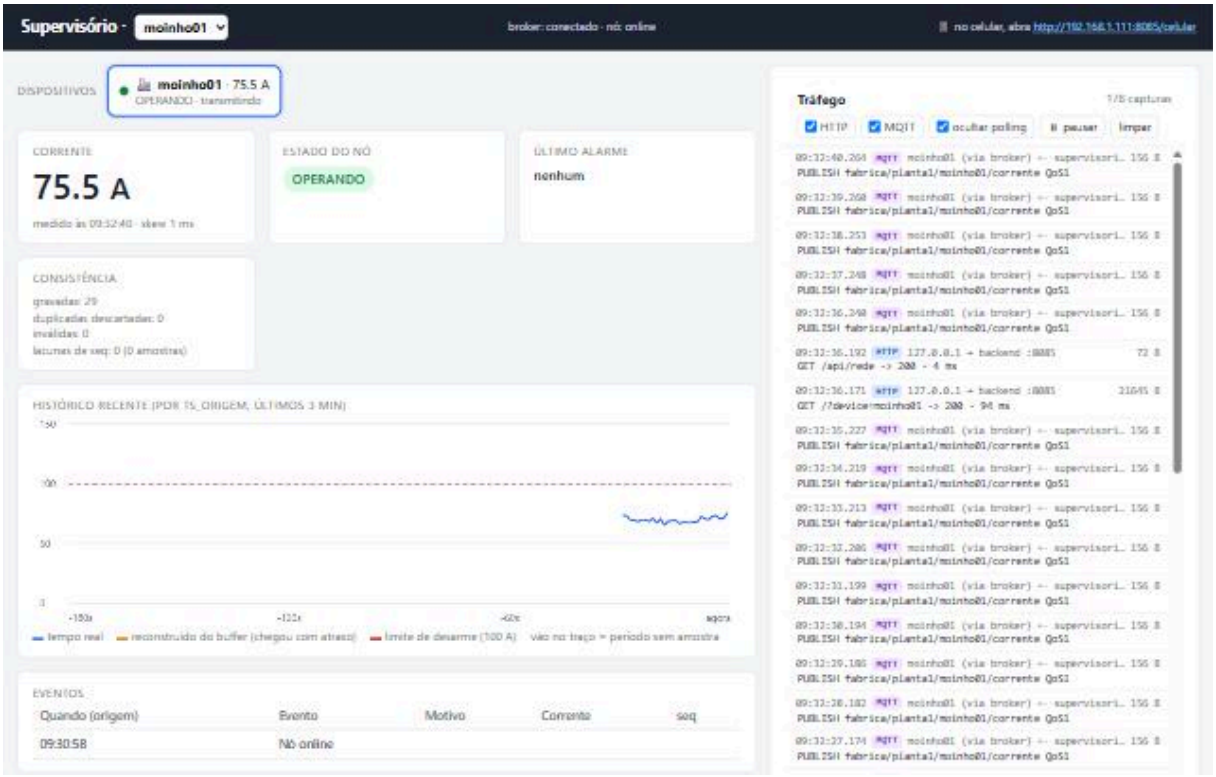


Figura 4 – Operação normal: corrente atual, estado OPERANDO e gráfico em tempo real. À direita, o painel de tráfego registra cada mensagem MQTT recebida pelo supervisório.

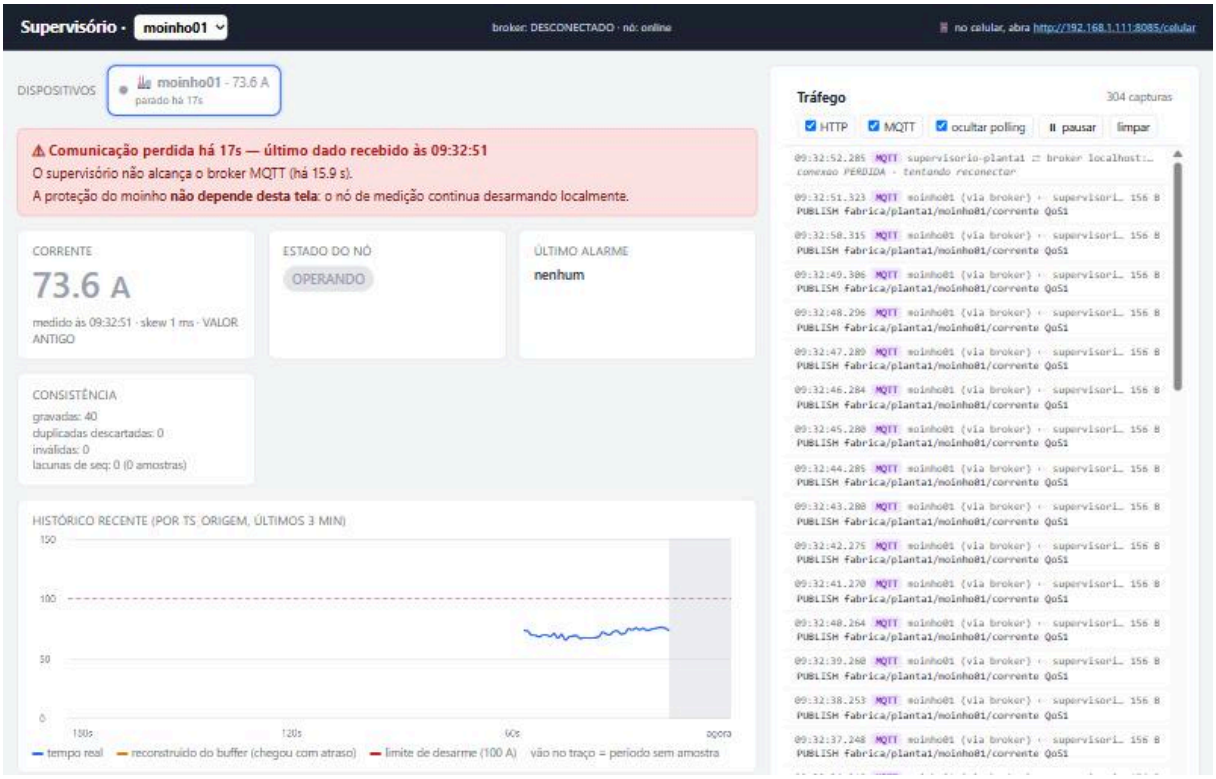


Figura 5 – Broker desligado: aviso de comunicação perdida, valores em cinza e trecho sem dados no gráfico. No mesmo instante, o nó já havia desarmado o moinho por sobrecorrente, sem rede.

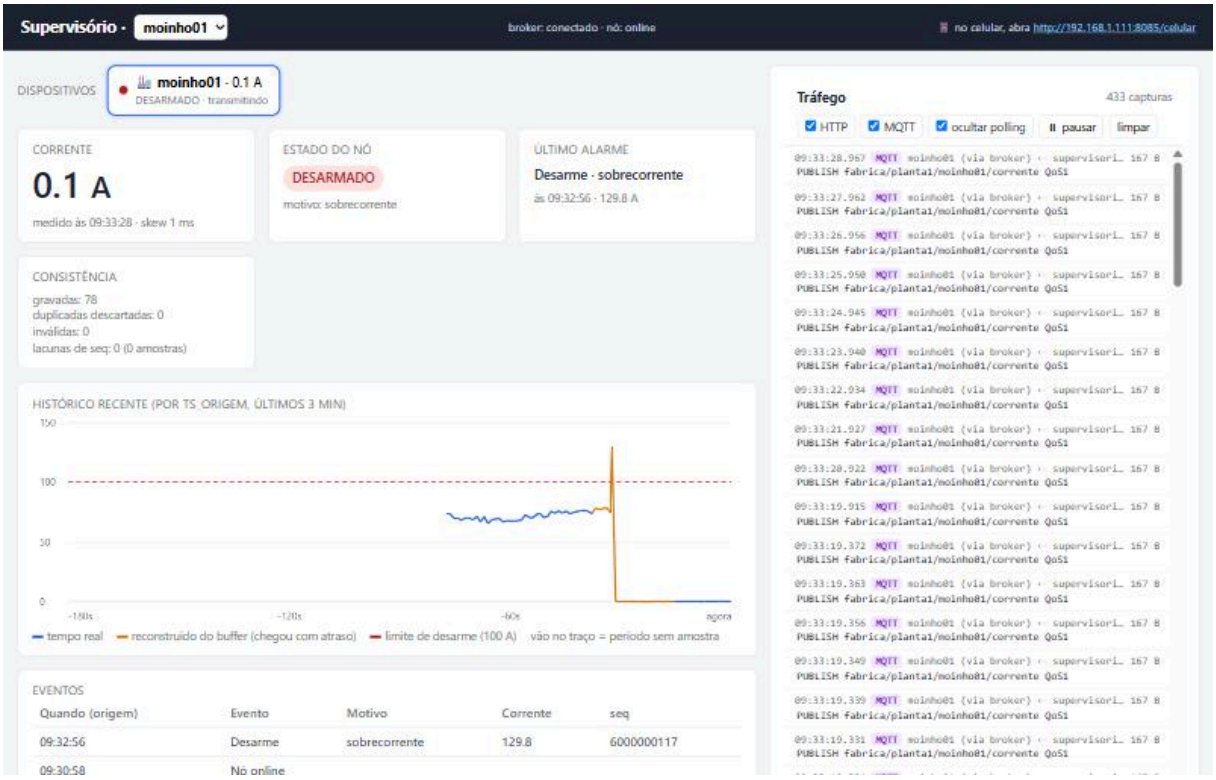


Figura 6 – Comunicação restabelecida: o trecho em laranja do gráfico veio do buffer do nó, e o desarme aparece registrado às 09:32:56, horário em que aconteceu, embora tenha chegado ao supervisório só às 09:33:19.



Figura 7 – Página que permite usar o celular no lugar do ESP32, com a mesma lógica de proteção, buffer e mensagens.

Tolerância a Falhas e Segurança

Nesta etapa, o fluxo da seção anterior (nó → MQTT → supervisório → banco → dashboard) passou a reagir quando os dados param de chegar, a registrar essas falhas e a proteger a comunicação contra acesso não autorizado. O código está no repositório https://github.com/GianMarcosBoaretto/moinho-mqtt, e o resumo técnico, em docs/TOLERANCIA_E_SEGURANCA.md.

Tempo máximo sem dados: 5 segundos. O nó envia uma leitura a cada 1 segundo, então 5 segundos correspondem a cinco leituras seguidas perdidas. Um limite menor acenderia o aviso a cada oscilação normal do Wi-Fi; um maior deixaria o operador tempo demais olhando um valor velho como se fosse atual. O critério usa o horário da medição (`ts_origem`), e não o de chegada: uma leitura antiga que chega atrasada continua sendo tratada como antiga, de modo que um dado velho nunca aparece como atual.

O que o usuário vê: passados os 5 segundos, o dashboard mostra o aviso "Comunicação perdida há X s — último dado recebido às HH:MM:SS", com a causa provável. O último valor continua na tela, em cinza e marcado como valor antigo, e o equipamento aparece como "parado há X s" (Figura 8).

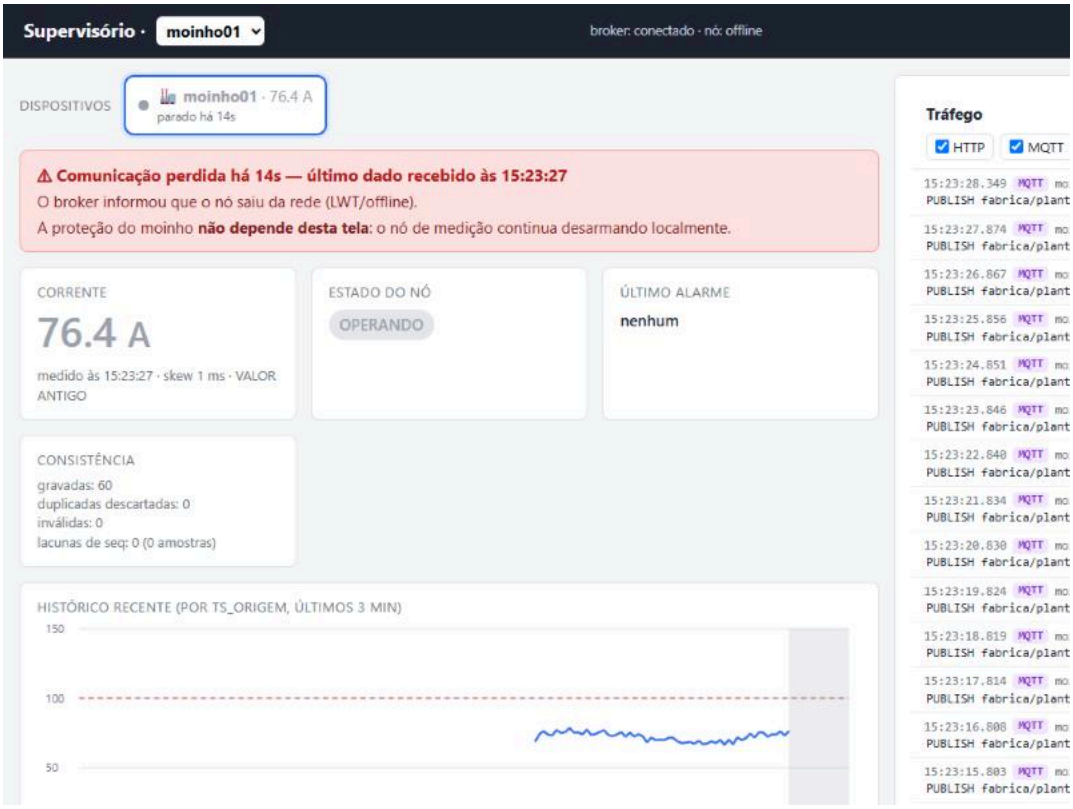


Figura 8 – Origem interrompida (simulador desligado): aviso de comunicação perdida há 14 s com o horário do último dado, valor mantido em cinza como valor antigo e nó marcado como offline pelo broker (LWT).

Teste de Falha e Registro em Log

Foram simuladas duas falhas: **falha de comunicação**, desligando o broker MQTT, e **falha da origem**, desligando o simulador. Nos dois casos o supervisório identificou a falta de dados em 5 segundos e exibiu o aviso. Quando os dados voltaram, o sistema retornou ao funcionamento normal sem reiniciar nada, porque o nó e o supervisório se reconectam sozinhos a cada 1 a 5 segundos.

O supervisório registra no arquivo `logs/supervisorio.log`, e também na tabela de eventos que aparece no dashboard, o momento em que os dados pararam e o momento em que voltaram, com a duração da falha:

```
15:23:32 [SEM DADOS] moinho01: dados pararam de chegar; ultimo dado medido
         em 18:23:27 UTC (ha 5.1s) - dashboard em modo degradado
15:23:54 [DADOS VOLTARAM] moinho01: dado atual de novo (26 s sem dados)
```

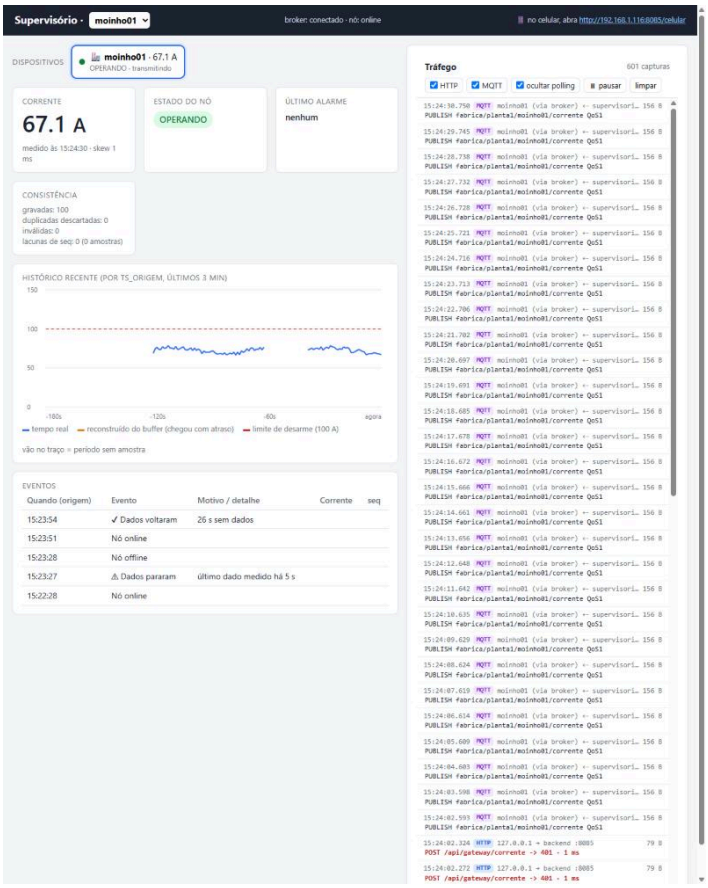


Figura 9 – Depois da volta: o gráfico mostra o intervalo sem dados, e a tabela de eventos registra "Dados pararam" (último dado medido há 5 s), nó offline, nó online e "Dados voltaram" (26 s sem dados). No painel de tráfego, à direita, aparecem em vermelho as tentativas de envio recusadas.

Um teste automatizado (`scripts/teste_falha_local.ps1`) repete os dois cenários e confere 25 itens, entre eles a detecção em até 5 segundos, o retorno automático, a ausência de lacunas no histórico e o registro no log. Todos passaram em 26/10/2026; o resultado está no

repositório.

Proteções Contra Falhas

O projeto implementa as quatro medidas sugeridas, e a principal é a primeira.

- **Funcionamento local sem depender da rede:** o desarme do moinho é decidido no nó. É a medida que mais pesa num processo industrial: uma queda de Wi-Fi não pode deixar o motor sem proteção.
- **Armazenamento temporário:** o nó guarda até 300 leituras (5 minutos) enquanto não há broker e as reenvia na volta, com o horário original.
- **Reconexão automática:** no nó e no supervisório, sem intervenção do operador.
- **Identificação de mensagens repetidas:** cada leitura tem um número de sequência, e o banco descarta a repetida pela chave (`device_id`, `seq`).

Segurança da Comunicação

Trecho vulnerável: o enlace entre o nó e o broker MQTT (Wi-Fi, porta 1883) é o mais crítico. Antes desta etapa, o broker aceitava conexões sem senha: qualquer pessoa na rede podia ler todas as leituras e publicar uma leitura falsa como se fosse o moinho, por exemplo "corrente normal" durante uma sobrecorrente real. A leitura falsa não desligaria nem ligaria o moinho, porque a proteção é local, mas enganaria o operador. A rota HTTP usada pelo celular tinha o mesmo problema, e todo o tráfego circulava sem criptografia (Figura 10).

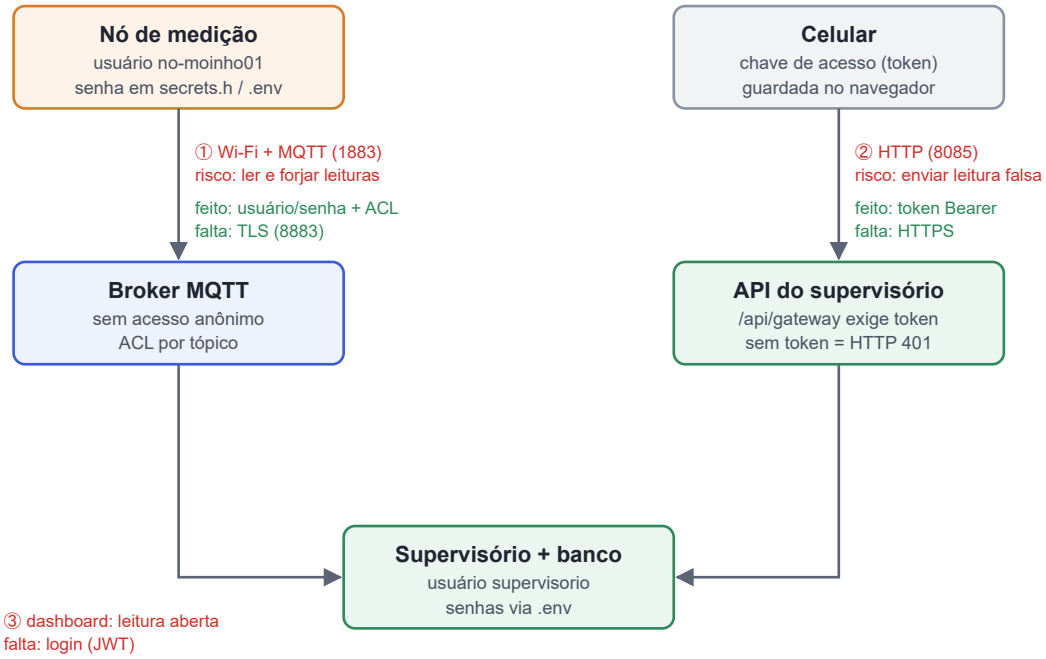


Figura 10 – Pontos de ataque do fluxo (em vermelho) e proteções aplicadas (em verde). Os dois enlaces de entrada exigem agora identificação; a criptografia (TLS/HTTPS) e o login no dashboard ficam como próximos passos.

Mecanismos que já existiam: a validação do conteúdo das mensagens (formato versionado, identificador do equipamento igual ao do tópico, valores dentro do domínio) e a independência da proteção em relação à rede.

Proteções implementadas: o broker deixou de aceitar conexões anônimas, e cada cliente tem usuário e senha próprios (no-moinho01 para o nó, supervisorio para o supervisório), guardados no broker apenas como hash. Uma lista de permissões por tópico (ACL) impede que o nó publique em nome de outro equipamento. A rota do celular passou a exigir uma chave de acesso no cabeçalho `Authorization: Bearer`; sem a chave, ou com a chave errada, a resposta é HTTP 401 e a tentativa é registrada no log (Figura 11). No teste, a conexão MQTT sem senha foi recusada pelo broker ("Not authorized") e uma publicação do nó em nome de outro moinho foi bloqueada.



Figura 11 – Tentativa de envio sem a chave de acesso: o supervisório responde HTTP 401 ("não autorizado") e o painel de tráfego registra a origem e a ferramenta usada.

Proteções indicadas para produção: TLS no MQTT (porta 8883, com certificado no broker e no ESP32), para que senhas e leituras não trafeguem em texto puro; HTTPS na API e no dashboard, por meio de um proxy reverso com certificado; login no dashboard com JWT de validade curta, já que hoje a leitura dos dados é aberta na rede; e uma rede separada (VLAN ou SSID próprio, com WPA2/WPA3) para os equipamentos de automação. TLS e HTTPS não foram implementados no protótipo para manter a demonstração simples, já que exigem gerar e instalar certificados.

Onde ficam as senhas e chaves: em um arquivo `.env` na raiz do projeto, gerado com valores aleatórios pelo script `scripts/preparar_segredos.py`, e, no ESP32, no arquivo `secrets.h`. Os dois estão no `.gitignore` e nunca vão para o repositório; no Git ficam apenas os modelos (`.env.exemplo` e `secrets.exemplo.h`). O código recebe as senhas por variáveis de ambiente, e cada processo recebe só a sua. O teste automatizado confere que nenhum desses arquivos está versionado. Se uma senha vazar, basta gerar um `.env` novo e reiniciar.

Comportamento do Sistema Durante uma Falha

A única ação automática do sistema é o desarme do moinho (abertura do relé ou contator) por sobrecorrente ou por falha do sensor, e ela não depende da rede. Com o Wi-Fi, o broker, o supervisório ou o banco fora do ar, o nó continua medindo a cada 100 ms e desarmando sozinho. Se o próprio sensor parar de responder, o nó desarma por segurança, já que sem medição não é possível saber se há sobrecorrente. A botoeira de emergência corta o relé eletricamente, mesmo com o ESP32 travado. O nó também não recebe comandos pela rede; o rearme é feito por um botão físico.

Assim, nenhuma decisão automática é tomada com base em dado desatualizado ou em comunicação indisponível: o desarme usa apenas a leitura feita no próprio nó, naquele instante. O que depende da rede é somente a informação mostrada ao operador, e essa parte avisa de forma explícita quando o dado está antigo, em vez de exibi-lo como atual.